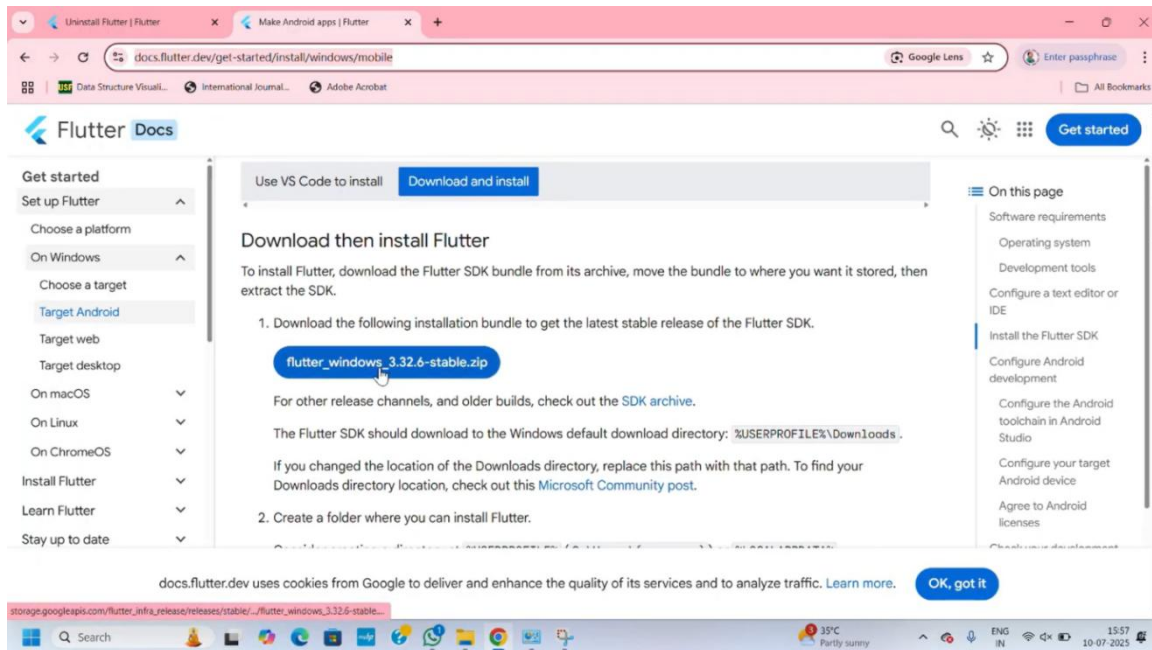


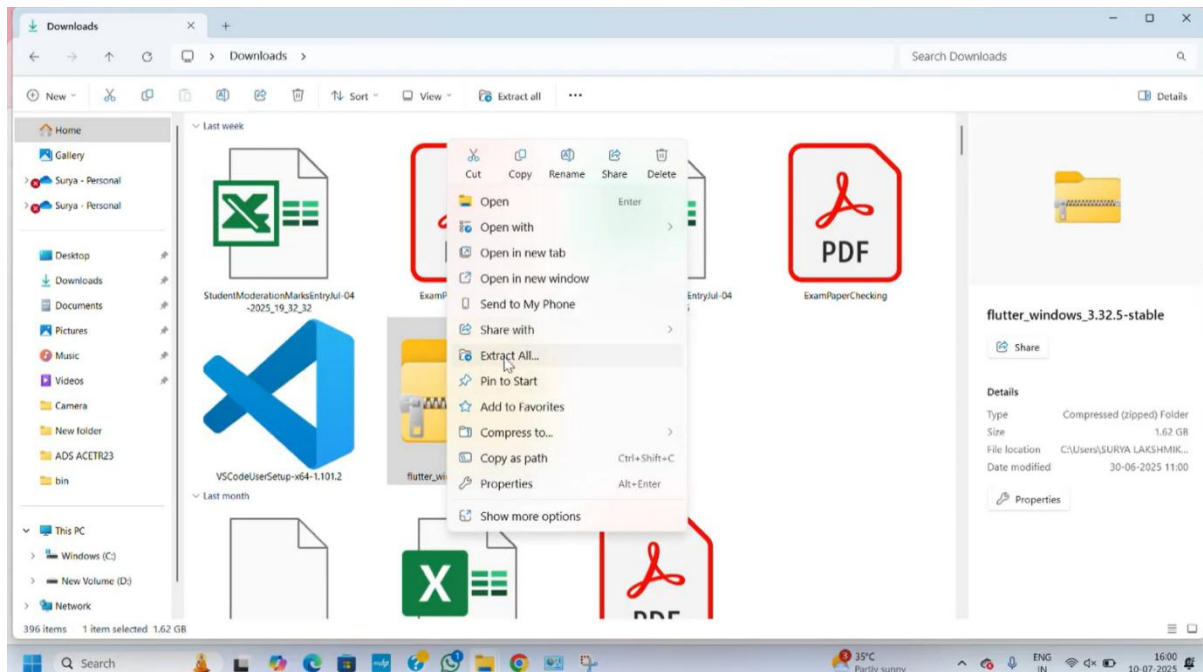
1.) Install Flutter and Dart SDK.

STEP-1: Download flutter SDK for windows OS from

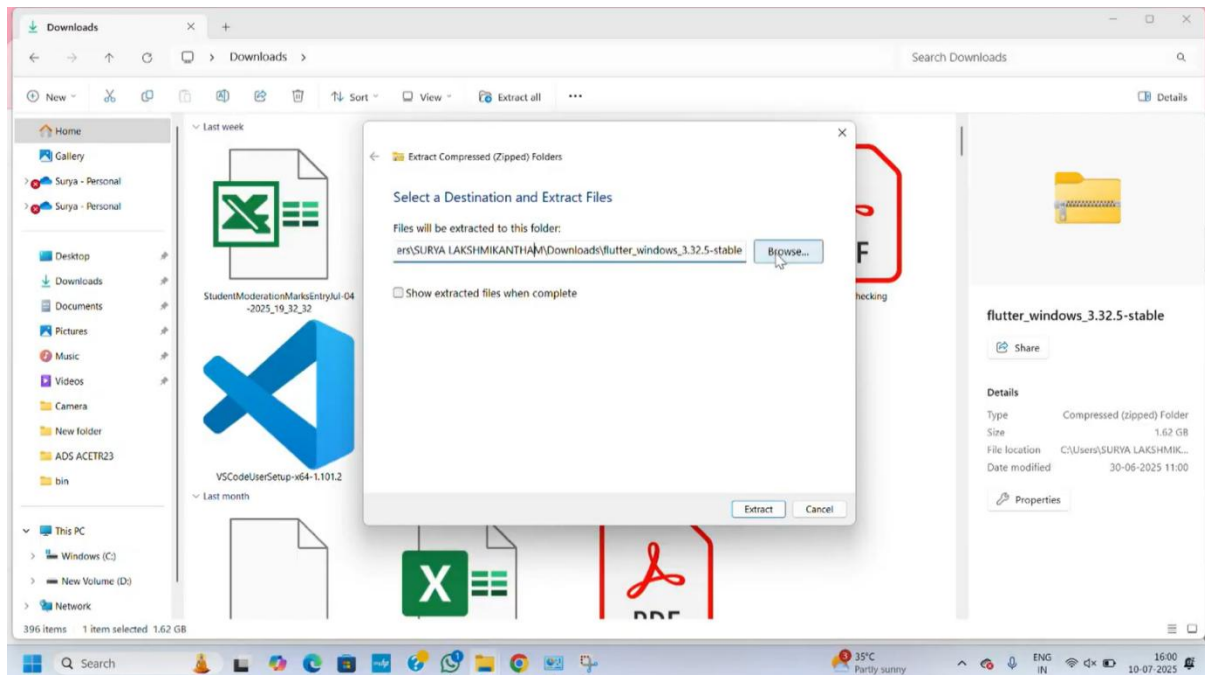
[<https://docs.flutter.dev/install/manual>] URL.



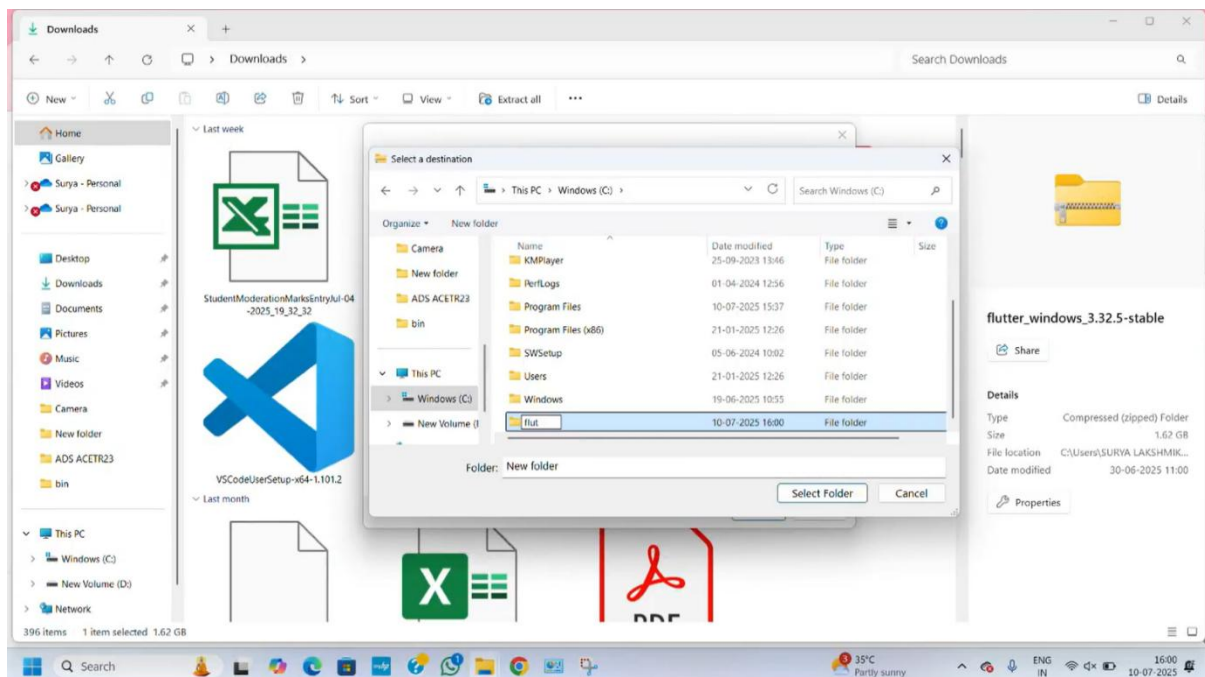
STEP-2: Extraction of flutter SDK into a specific folder.



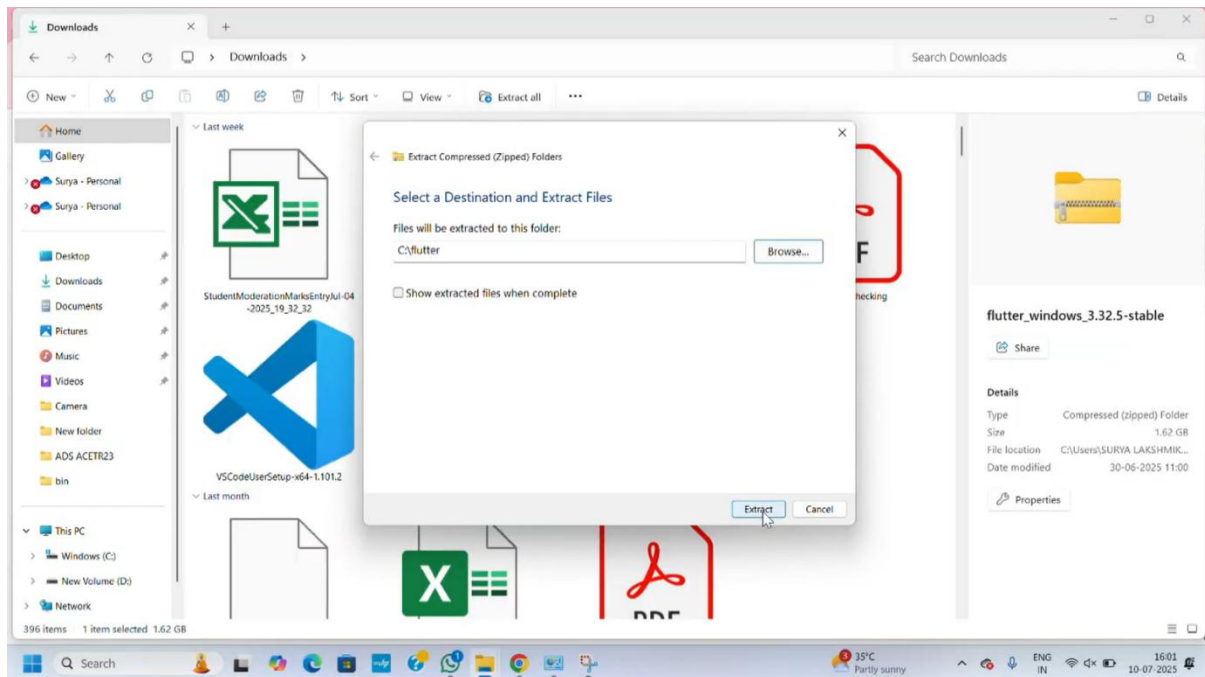
STEP-3: Once it is downloaded, extraction must be done. Click on the Zip file to extract..



STEP-4: we have to select the folder



STEP-5: Once, the folder is created, extract the contents, by selecting the folder.



STEP-6: to check whether the flutter SDK is successfully installed, go to command prompt and type **flutter --version** and click enter

```
Command Prompt - flutter
Microsoft Windows [Version 10.0.26100.4351]
(c) Microsoft Corporation. All rights reserved.

C:\Users\SURYA LAKSHMIKANTHAM>flutter --version
'flutter' is not recognized as an internal or external command,
operable program or batch file.

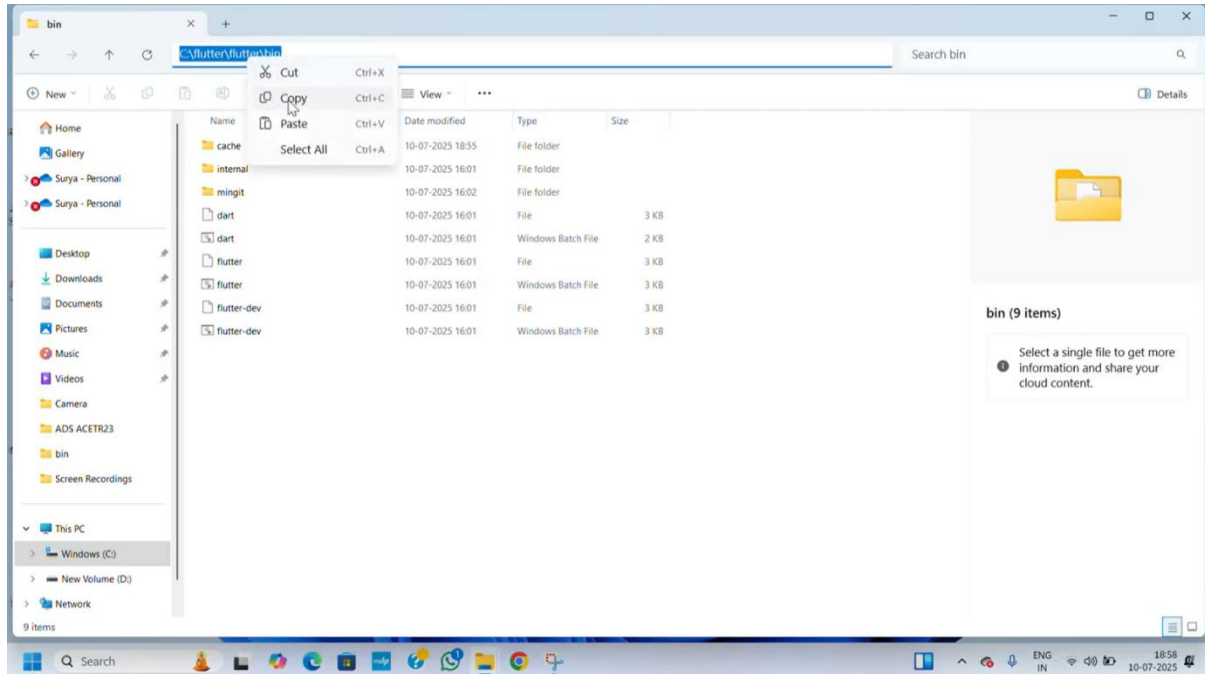
C:\Users\SURYA LAKSHMIKANTHAM>flutter --version
Building flutter tool...
Running pub upgrade...
Resolving dependencies... (1.2s)
Downloading packages...
Got dependencies.
Flutter 3.32.5 • channel stable • https://github.com/flutter/flutter.git
Framework • revision fcf2c11572 (2 weeks ago) • 2025-06-24 11:44:07 -0700
Engine • revision dd93de6fb1 (2 weeks ago) • 2025-06-24 07:39:37 -0700
Tools • Dart 3.8.1 • DevTools 2.45.1

C:\Users\SURYA LAKSHMIKANTHAM>
```

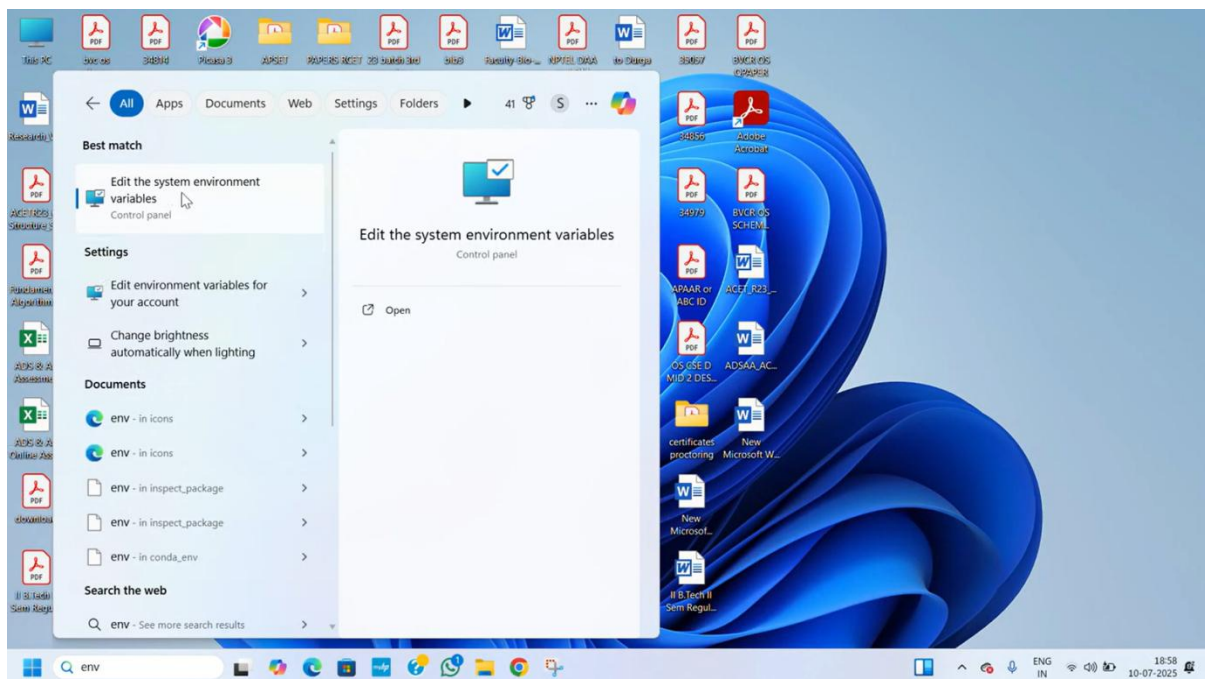
Setting Environmental Variable

STEP-1: To set up the path, go to folder created on c-drive and then move to the flutter and then to the bin.

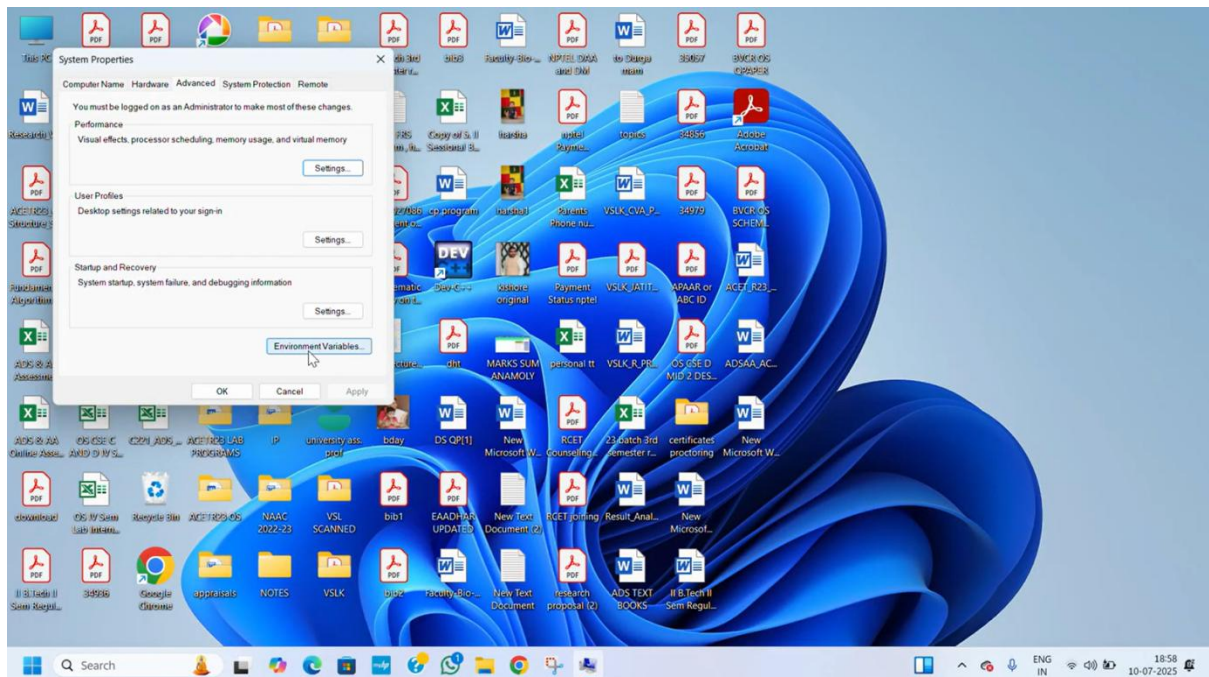
Now copy the path `c:\FLUTTER\flutter\bin`



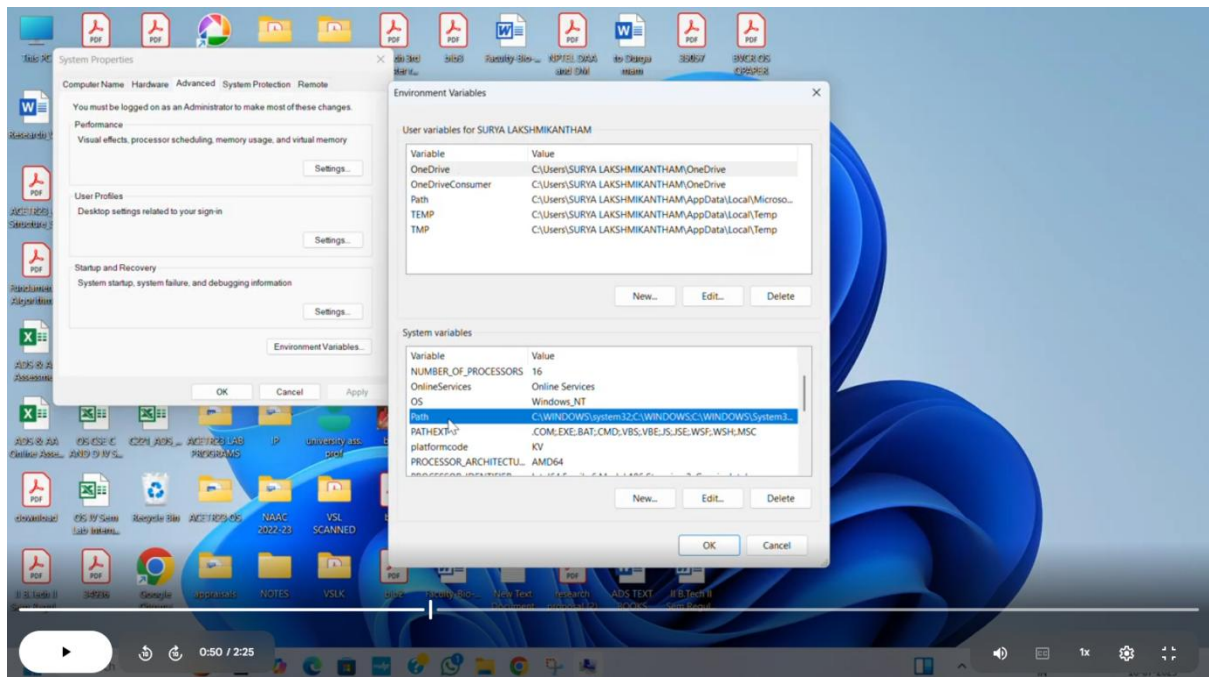
STEP-2: Then go to “Edit the system Environment Variables” control panel



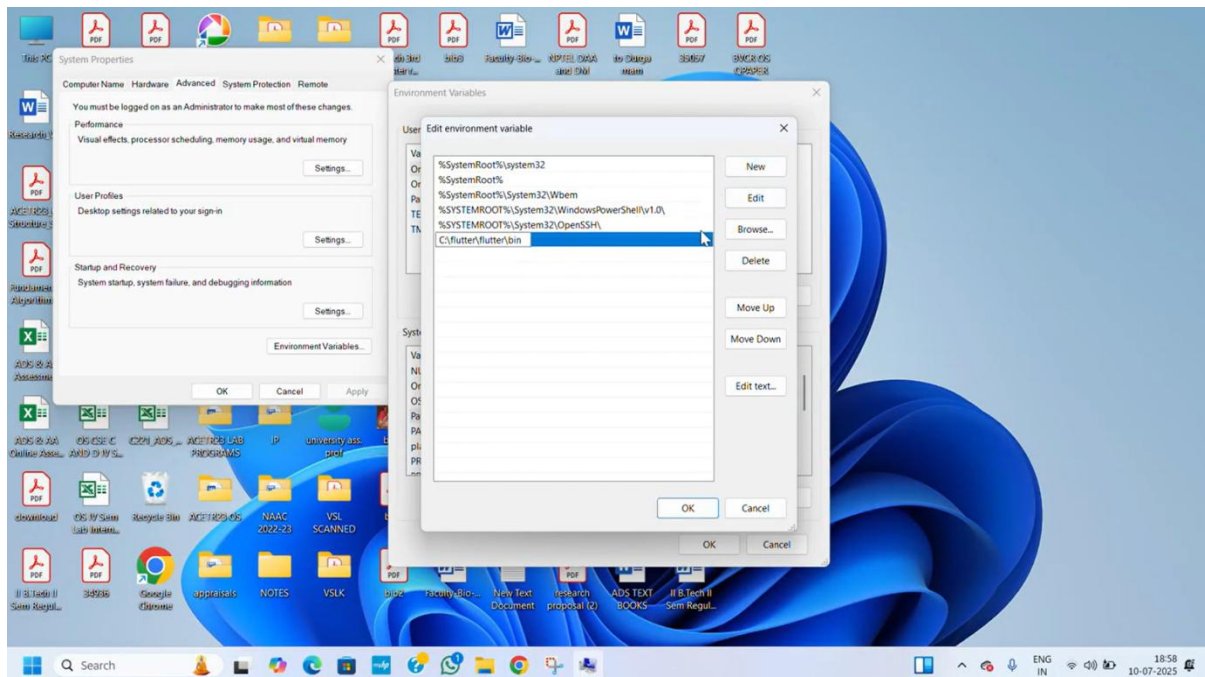
STEP-3: Then click on Environment Variables.



STEP-4: Then move to System variables. Double click on Path and there copy the path link.



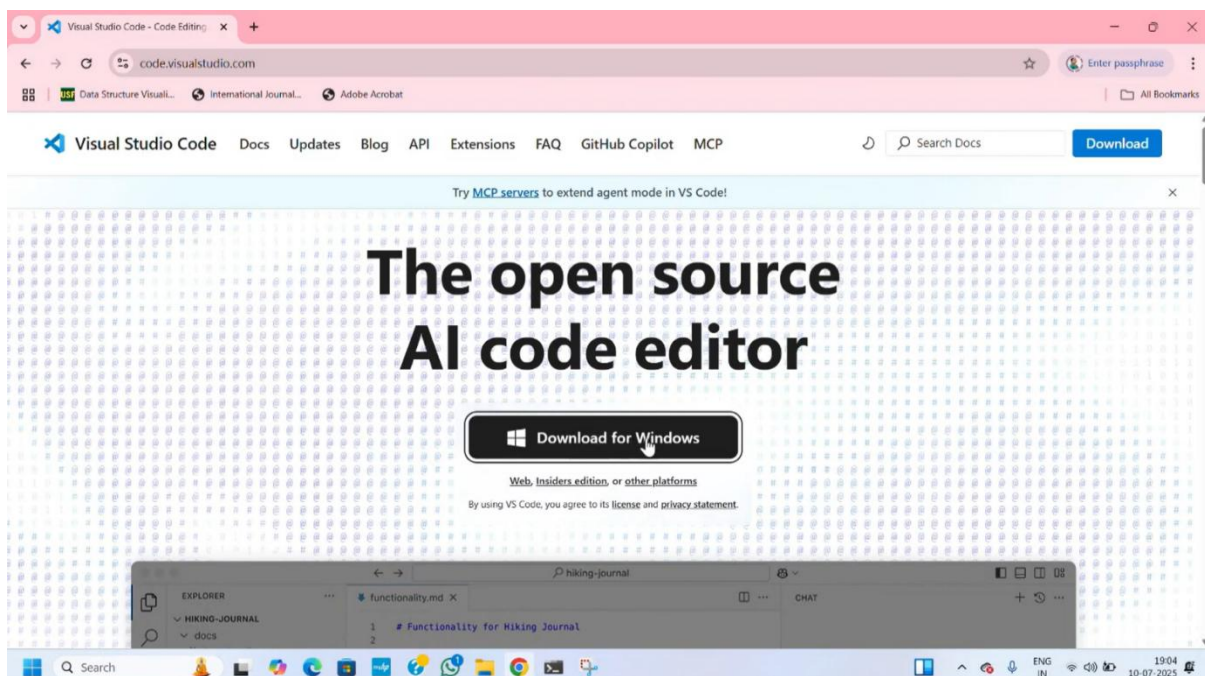
STEP-5: Click on new, then copy the path link.



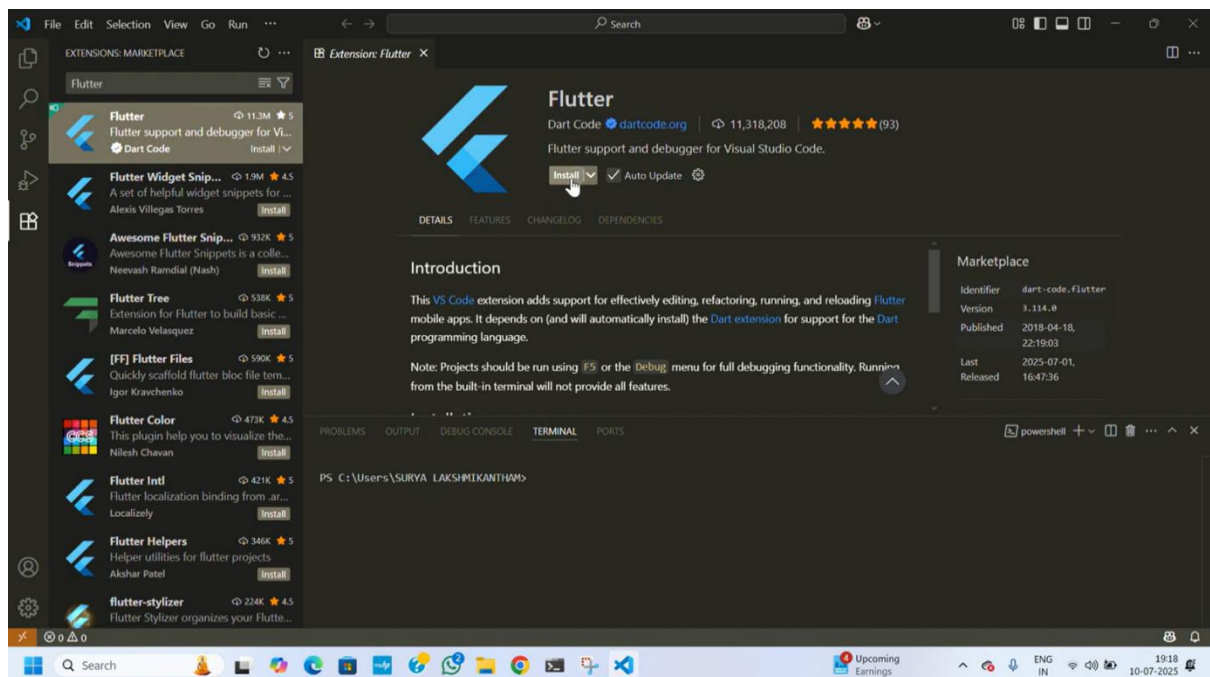
STEP-6: Come back by clicking OK everywhere.

Downloading Flutter and Dart Extensions

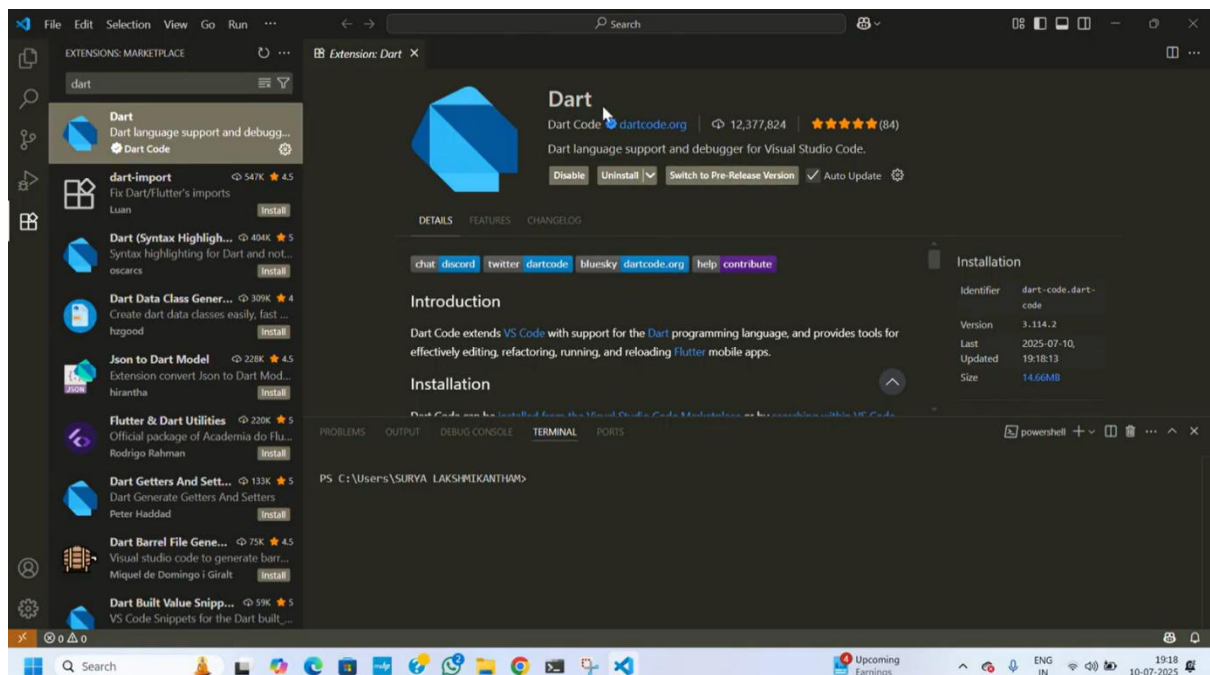
STEP-1: First install the VISUAL STUDIO CODE from user search engine.



STEP-2: Once it is installed, search for the Flutter extension and install it.



STEP-3: Search Dart extension and install it.



1b)

Write a dart program to illustrate basic data types and print function.

Code:

```
void main() {  
  // Integer variable  
  int count = 9;  
  // Boolean variable  
  bool flag = true;  
  // Double (floating point) variable  
  double marks = 55.78;  
  // String variable  
  String name = 'Amar';  
  // var automatically takes String type  
  var hai = "Good Morning";  
  // Printing using string interpolation  
  print("$hai! My name is $name with aggregate $marks");  
  print("flag is $flag and count is $count");  
}
```

OUTPUT:

Good Morning! My name is Amar with aggregate 55.78

flag is true and count is 9

Write a dart program to read multiple variables of different data types from keyboard and print them.

Code:

```
import 'dart:io';

void main() {

  print('Enter age, height, and name :');

  // Take a line of input, split by spaces, store as List<String>
  List<String> inputs = stdin.readLineSync()!.split(' ');

  // Convert to proper data types

  int age = int.parse(inputs[0]);    // First input → int
  double height = double.parse(inputs[1]); // Second input → double
  String name = inputs[2];          // Third input → String

  // Print values

  print('Age: $age, Height: $height, Name: $name');
}
```

OUTPUT

Enter age, height, and name :

30 180 Ram

Age:30, Height:180.0, Name:Ram

Write a dart program on logical operators

code:

```
void main() {  
    bool a = true;  
    bool b = false;  
  
    // logical AND (&&)  
    print('a && b = ${a && b}');  
  
    // logical OR (||)  
    print('a || b = ${a || b}');  
  
    // logical NOT (!)  
    print('!a = ${!a}');  
    print('!b = ${!b}');  
  
    // combining expressions  
  
    int x = 10;  
    int y = 20;  
  
    print('(x > 5) && (y > 30) = ${ (x > 5) && (y > 30) }');  
    print('(x > 5) && (y < 30) = ${ (x > 5) && (y < 30) }');  
    print('(x < 5) && (y < 30) = ${ (x < 5) && (y < 30) }');  
    print('(x < 5) || (y > 15) = ${ (x < 5) || (y > 15) }');  
}
```

OUTPUT:

a && b = false

a || b = true

!a = false

!b = true

(x>5) && (y>30) = false

(x>5) && (y<30) = true

(x<5) && (y<30) = false

(x<5) || (y>15) = true

Write a Dart program to read number from user, and find whether it is negative or positive. If positive , check whether it is even or odd.

PROGRAM :

```
import 'dart:io';

void main() {
  print("Enter a number:");
  int number = int.parse(stdin.readLineSync());
  // Simple if statement
  if (number > 0) {
    print("The number is positive.");
    if (number % 2 == 0) {
      print("The number is even.");
    } else {
      print("The number is odd.");
    }
  } else if (number < 0) {
    print("Negative number");
  } else {
    print("Zero");
  }
}
```

OUTPUT :

```
PS D:\CSE C FLUTTER\dart_application_3\bin> dart .\dart_application_3.dart
Enter a number:
9
The number is positive.
The number is odd.
PS D:\CSE C FLUTTER\dart_application_3\bin> dart .\dart_application_3.dart
Enter a number:
0
Zero
```

WRITE A DART PROGRAM TO SIMULATE SIMPLE CALCULATOR

```
import 'dart:io';

void main() {

  //Simple Calculator
  print("\n--- Simple Calculator ---");
  print("Enter first number:");
  double num1 = double.parse(stdin.readLineSync());

  print("Enter operator (+, -, *, /):");
  String op = stdin.readLineSync();

  print("Enter second number:");
  double num2 = double.parse(stdin.readLineSync());

  double result;

  switch (op) {
    case '+':
      result = num1 + num2;
      print("Result: $result");
      break;
    case '-':
      result = num1 - num2;
      print("Result: $result");
      break;
    case '*':
      result = num1 * num2;
      print("Result: $result");
      break;
    case '/':
      if (num2 != 0) {
        result = num1 / num2;
        print("Result: $result");
      } else {
        print("Error: Division by zero");
      }
      break;
    default:
      print("Invalid operator");
  }
}
```

OUTPUT :

--- Simple Calculator ---

Enter first number:

5

Enter operator (+, -, *, /):

+

Enter second number:

3

Result: 8.0

2 a) Explore various Flutter widgets (Text, Image, Container, etc.).

Creating Flutter Widget Text :

```
import 'package:flutter/material.dart';

void main() {

  runApp(MyApp());

}

class MyApp extends StatelessWidget {

  @override

  Widget build(BuildContext context) {

    return MaterialApp(

      home: Scaffold(

        appBar: AppBar(

          title: Text("This is AppBar"),

          titleTextStyle: TextStyle(

            fontSize: 45,

            fontWeight: FontWeight.w600,

            color: Colors.pink,

          ),

          backgroundColor: Colors.amberAccent,

        ),

      ),

    );

  }

}
```

ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY

- `runApp(MyApp())`: Launches your app.
- `MaterialApp`: Wraps your app in Material Design.
- `Scaffold`: Provides basic layout like `AppBar`, `body`.
- `AppBar`: Top bar with title.
- `Text`: Displays a string on screen.
- `Center`: Centers the text on the screen.

OUTPUT :



Creating Flutter Widget Image:

```
import 'package:flutter/material.dart';

void main() {

  runApp(MyApp());

}

class MyApp extends StatelessWidget {

  @override

  Widget build(BuildContext context) {

    return MaterialApp(

      home: Scaffold(

        appBar: AppBar(title: Text("Inserting image in App")),

        body: Center(child: Image.asset('assets/vangod_spirit.jpeg')),

      ),

    );

  }

}
```

Steps to Add Local Image:

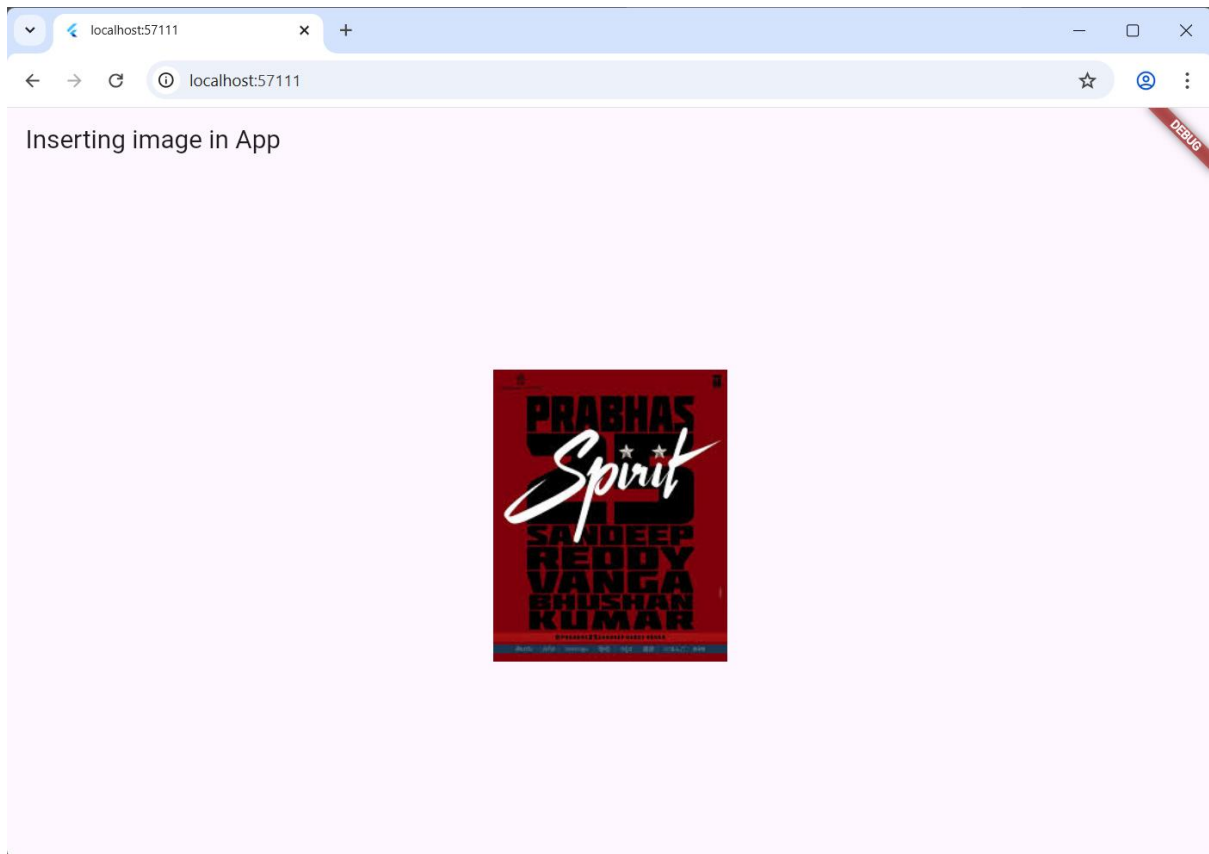
1. Create a folder in your project with name **assets**
2. Place your image (e.g., vangod_spirit.jpeg) inside that folder.
3. Edit your **pubspec.yaml** file in the project folder :

flutter:

assets:

- assets/vangod_spirit.jpeg

OUTPUT :



Creating Flutter Widget Container:

```
import 'package:flutter/material.dart';

void main() {

  runApp(MyApp());

}

class MyApp extends StatelessWidget {

  @override

  Widget build(BuildContext context) {

    return MaterialApp(

      home: Scaffold(

        appBar: AppBar(title: Text('Container Example')),

        body: Center(

          child: Container(

            width: 150,

            height: 150,

            padding: EdgeInsets.all(50),

            margin: EdgeInsets.all(50),

            decoration: BoxDecoration(

              color: Colors.blue[200],

              borderRadius: BorderRadius.circular(22),

              border: Border.all(color: Colors.blue, width: 12),

            ),

            child: Center(

              child: Container(color: Colors.orange, width: 100, height: 100),
```

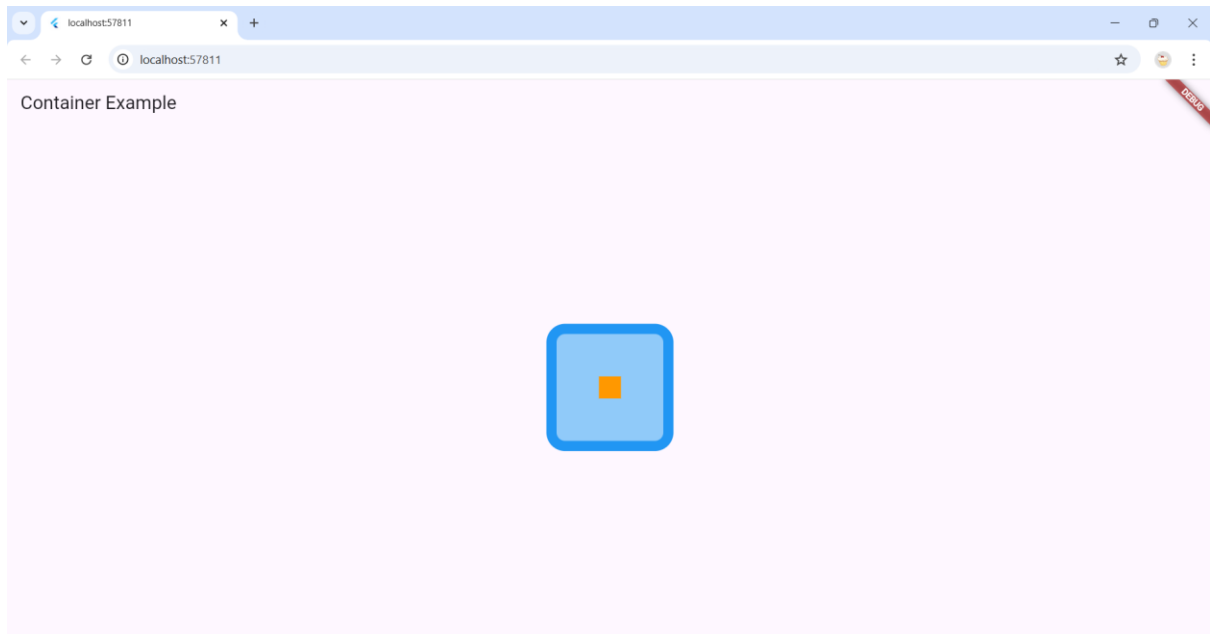
```

    ),
    ),
    ),
    ),
);
}
}

```

Attribute	Purpose
width	Sets the container's width (in logical pixels)
height	Sets the container's height
color	Background color (use directly only if no decoration is used)
alignment	Aligns the child widget inside the container (e.g., Alignment.center)
padding	Space inside the container, around its child
margin	Space outside the container, around its border
child	The single widget placed inside the container
decoration	Used for styling like background image, gradient, border, shadows

OUTPUT :



2 b) Implement different layout structures using Row, Column, and Stack widgets.

Implementing Row (layout Structure) Widget :

```
import 'package:flutter/material.dart';

void main() {

  runApp(MyApp());

}

class MyApp extends StatelessWidget {

  @override

  Widget build(BuildContext context) {

    return MaterialApp(

      home: Scaffold(

        appBar: AppBar(title: Text('Demonstrating row Layout')),

        body: Center(

          child: Row(

            mainAxisAlignment: MainAxisAlignment.spaceEvenly,

            crossAxisAlignment: CrossAxisAlignment.center,

            children: [

              Container(

                height: 80,

                width: 80,

                color: Colors.blueAccent,

                child: Text("Box1"),

              ),

              Container(
```

```
        height: 80,

        width: 80,

        color: Colors.yellow,

        child: Text("Box2"),

    ),

    Container(

        height: 80,

        width: 80,

        color: Colors.lightGreen,

        child: Text("Box3"),

    ),

  ],

),

),

),

);

}

}
```

- Row: Lays out its children **horizontally**
- mainAxisAlignment.spaceEvenly: Distributes the children **with equal spacing**
- crossAxisAlignment.center: Aligns all boxes **vertically in the center**
- Three Containers with different background colors and labels

OUTPUT :



Implementing Row (layout Structure) Widget :

```
import 'package:flutter/material.dart';

void main() {

  runApp(MyApp());

}

class MyApp extends StatelessWidget {

  @override

  Widget build(BuildContext context) {

    return MaterialApp(

      home: Scaffold(

        appBar: AppBar(title: Text("Demonstrating Column Layout")),

        body: Column(

          mainAxisAlignment: MainAxisAlignment.start,

          crossAxisAlignment: CrossAxisAlignment.start,

          children: [

            Container(

              height: 55,

              width: 55,

              child: Text("Box1"),

              color: Colors.blueAccent,

            ),

            Container(

              height: 55,

              width: 55,
```

```
    child: Text("Box2"),  
    color: Colors.pinkAccent,  
  ),  
  Container(  
    height: 55,  
    width: 55,  
    child: Text("Box3"),  
    color: Colors.tealAccent,  
  ),  
],  
,  
,  
);  
}  
}
```

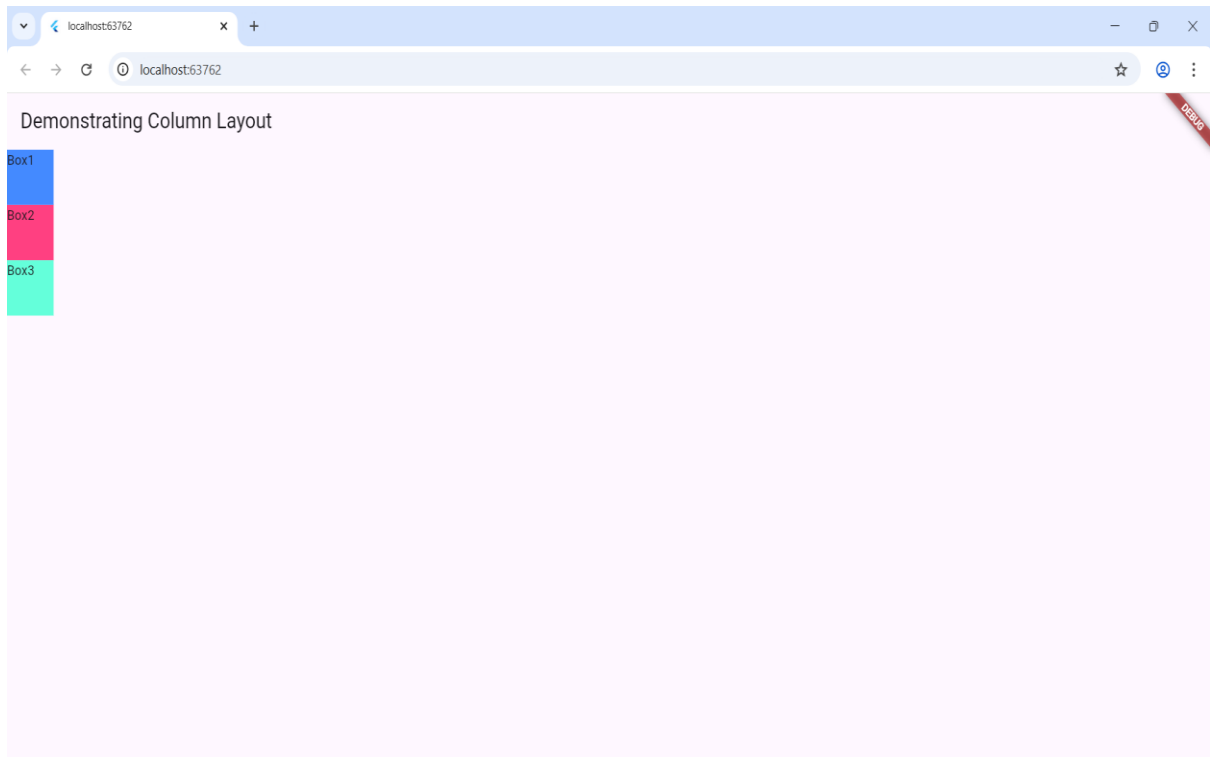
mainAxisAlignment

- Defines how children are placed along the main axis.
- For a Row, the main axis is **horizontal**.
- For a Column, the main axis is **vertical**.]

crossAxisAlignment

- Defines how children are placed along the cross axis.
- For a Row, the cross axis is **vertical**.
- For a Column, the cross axis is **horizontal**.

OUTPUT :



Implementing Row (layout Structure) Widget :

```
import 'package:flutter/material.dart';

void main()

{

runApp(MyApp());

}

class MyApp extends StatelessWidget {

  @override

  Widget build(BuildContext context) {

    return MaterialApp(

      home: Scaffold(

        appBar: AppBar(title: Text('Stack Layout Example')),

        body: Center(

          child: Stack(

            alignment: Alignment.center,

            children: [

              Container(

                width: 200,

                height: 200,

                color: Colors.red,

                child: Center(child: Text('Box 1')),

              ),

              Container(

                width: 150,
```

```
        height: 150,

        color: Colors.green,

        child: Center(child: Text('Box 2')),

    ),

    Container(

        width: 100,

        height: 100,

        color: Colors.blue,

        child: Center(child: Text('Box 3')),

    ),

  ],

),

),

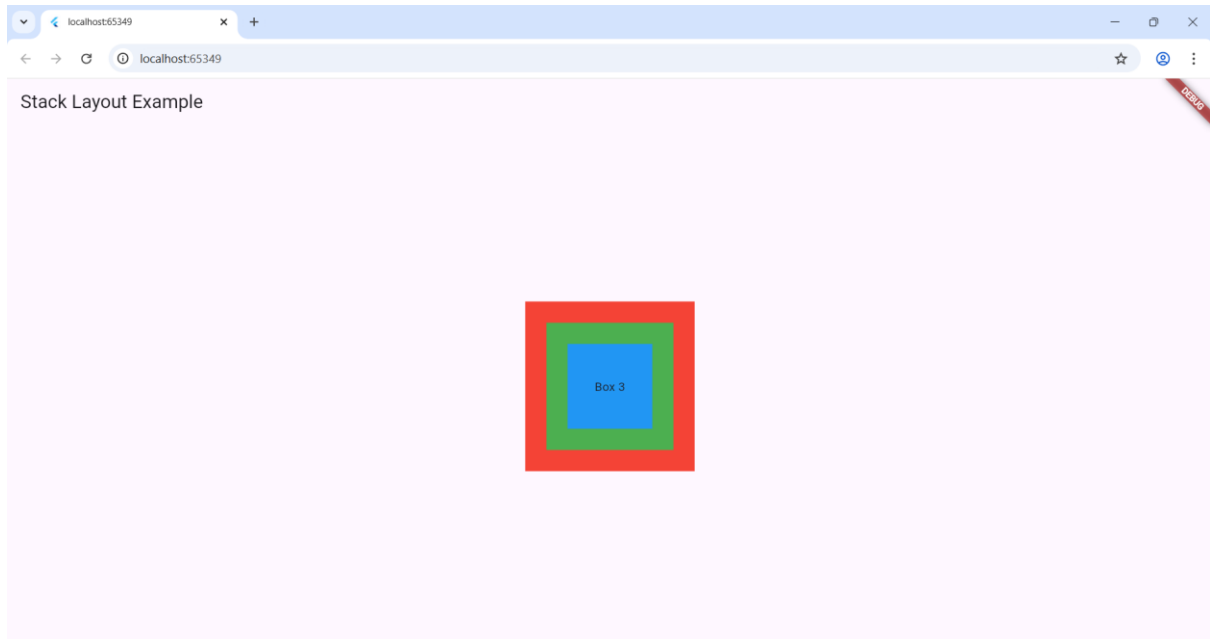
),

);

}

}
```

OUTPUT :



3 a) Design a responsive UI that adapts to different screen sizes.

Beginner-friendly responsive UI design in Flutter that adapts gracefully to different screen sizes (like mobile and tablet). It uses:

- MediaQuery to detect screen width
- LayoutBuilder to switch layout dynamically
- Flexible for resizing elements
- Clean UI using Containers and Text widgets

PROGRAM :

```
import 'package:flutter/material.dart';

void main()

{

  runApp(MyResponsiveApp());

}

class MyResponsiveApp extends StatelessWidget {

  @override

  Widget build(BuildContext context) {

    return MaterialApp(

      title: 'Responsive UI',

      theme: ThemeData(primarySwatch: Colors.blue),

      home: ResponsiveHomePage(),

    );

  }

}

class ResponsiveHomePage extends StatelessWidget {

  @override
```

```
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(title: Text('Responsive UI Example')),  
    body: LayoutBuilder(  
      builder: (context, constraints) {  
        if (constraints.maxWidth < 600) {  
          //Small screen: use vertical layout  
          return _buildVerticalLayout();  
        } else {  
          //Large screen: use horizontal layout  
          return _buildHorizontalLayout();  
        }  
      },  
    ),  
  );  
}
```

//Mobile View

```
Widget _buildVerticalLayout() {  
  return Column(  
    children: [  
      _buildBox(Colors.red, 'Box 1'),  
      SizedBox(height: 10),  
      _buildBox(Colors.green, 'Box 2'),  
    ],  
  );  
}
```

```
        SizedBox(height: 10),  
        _buildBox(Colors.blue, 'Box 3'),  
      ],  
    );  
  }
```

//  **Tablet/Desktop View**

```
Widget _buildHorizontalLayout() {  
  return Row(  
    mainAxisAlignment: MainAxisAlignment.spaceEvenly,  
    children: [  
      Flexible(child: _buildBox(Colors.red, 'Box 1')),  
      Flexible(child: _buildBox(Colors.green, 'Box 2')),  
      Flexible(child: _buildBox(Colors.blue, 'Box 3')),  
    ],  
  );  
}
```

// **Reusable UI block**

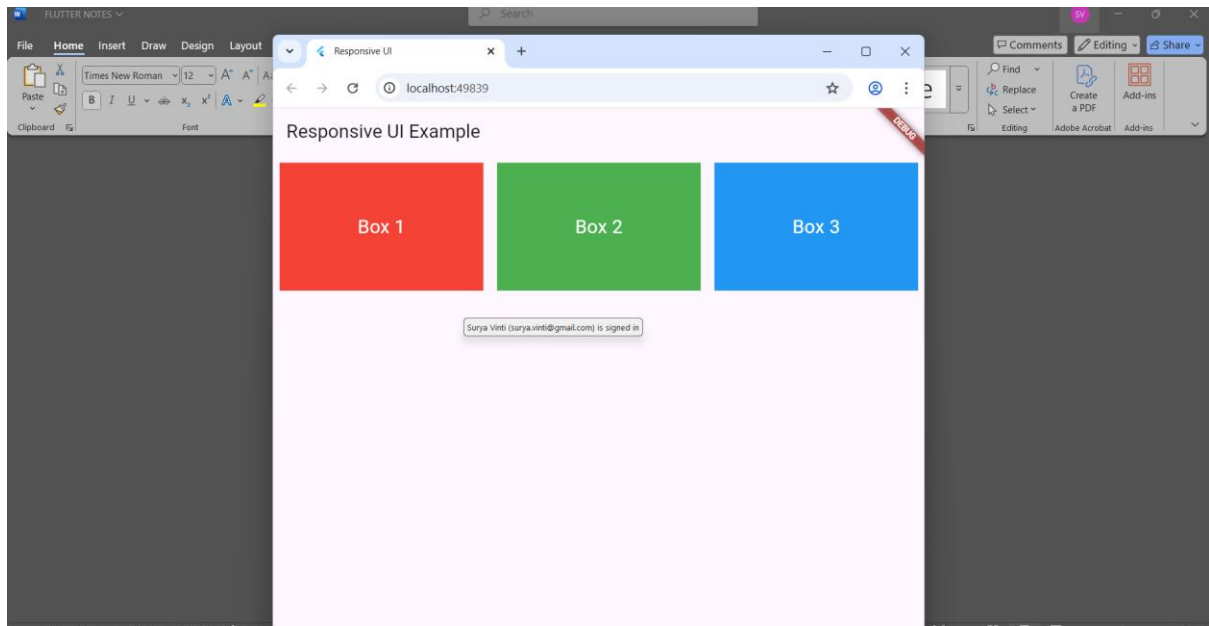
```
Widget _buildBox(Color color, String label) {  
  return Container(  
    height: 150,  
    margin: EdgeInsets.all(8),  
    color: color,
```

```
child: Center(  
  child: Text(  
    label,  
    style: TextStyle(fontSize: 22, color: Colors.white),  
  ),  
),  
);  
}  
}
```

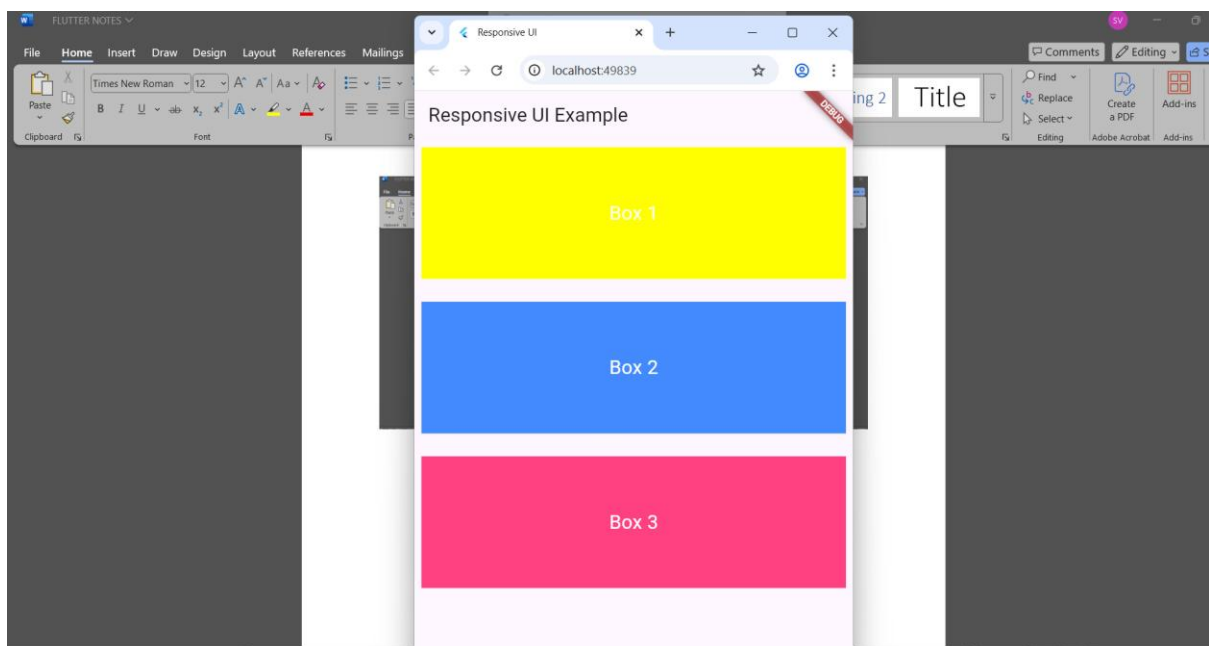
OUTPUT :



DESKTOP/TAB VIEW



DESKTOP/TAB VIEW



MOBILE VIEW

3 b) Implement media queries and breakpoints for responsiveness.

```
import 'package:flutter/material.dart';

void main()

{
runApp(MyResponsiveApp());
}

class MyResponsiveApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Responsive UI with MediaQuery',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: ResponsiveHomePage(),
    );
  }
}

class ResponsiveHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    // 📏 Screen width from MediaQuery
    double screenWidth = MediaQuery.of(context).size.width;

    Widget content;

    // 📱 Check breakpoints
    if (screenWidth < 600) {
      content = _buildMobileLayout();
    } else if (screenWidth >= 600 && screenWidth < 1024) {
      content = _buildTabletLayout();
    }
  }
}
```

```
    } else {  
        content = _buildDesktopLayout();  
    }  
  
    return Scaffold(  
        appBar: AppBar(title: Text('Responsive Layout')),  
        body: content,  
    );  
}
```

// 📱 Mobile layout (column)

```
Widget _buildMobileLayout() {  
    return Column(  
        children: [  
            _buildBox(Colors.red, 'Mobile Box 1'),  
            _buildBox(Colors.green, 'Mobile Box 2'),  
            _buildBox(Colors.blue, 'Mobile Box 3'),  
        ],  
    );  
}
```

// 📺 Tablet layout (grid-like)

```
Widget _buildTabletLayout() {  
    return Padding(  
        padding: const EdgeInsets.all(8.0),  
        child: Wrap(  
            spacing: 10,  
            runSpacing: 10,  
            children: [  
                _buildBox(Colors.red, 'Tablet Box 1'),  
            ],  
        ),  
    );  
}
```

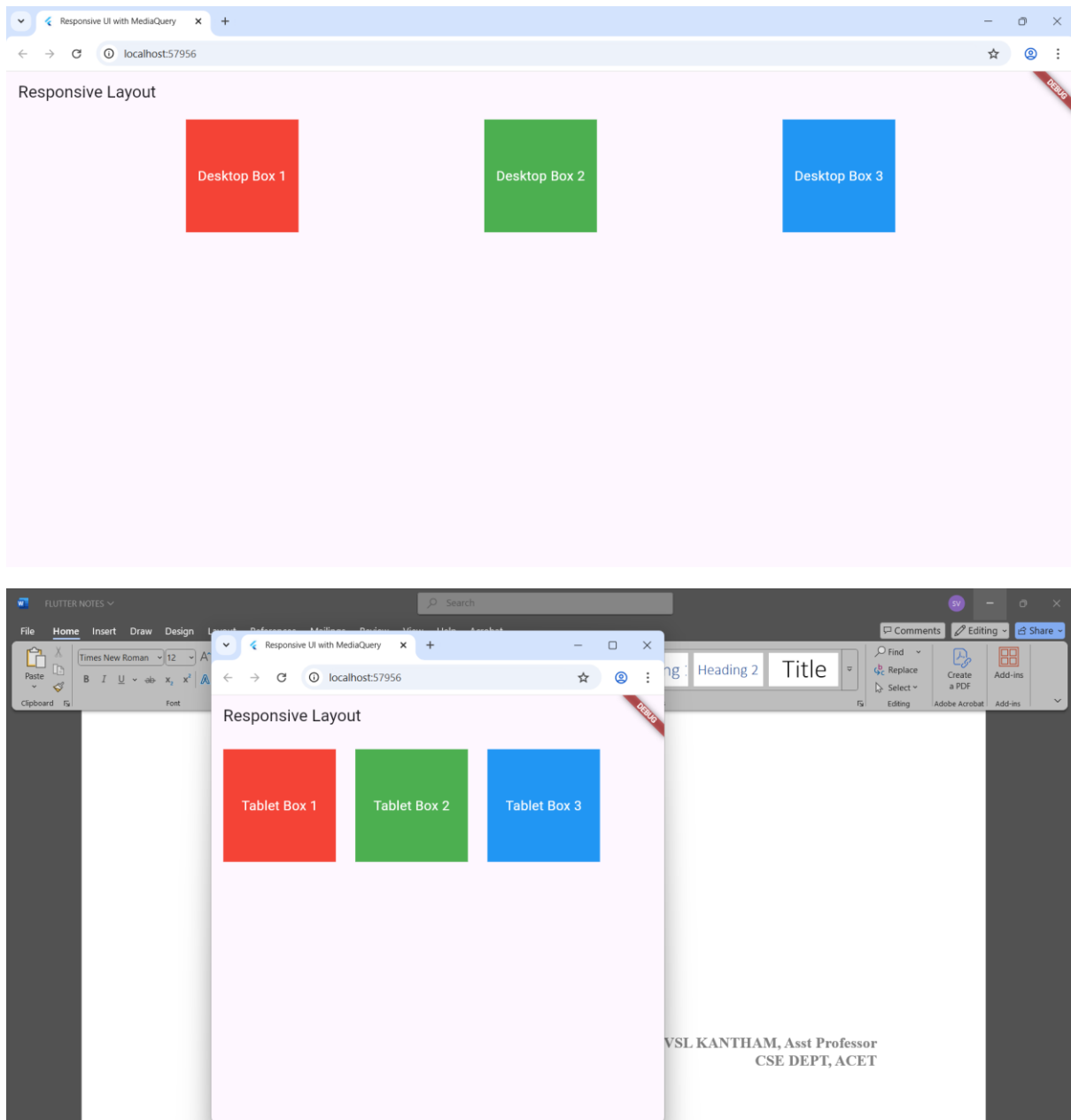
```
        _buildBox(Colors.green, 'Tablet Box 2'),
        _buildBox(Colors.blue, 'Tablet Box 3'),
    ],
),
);
}

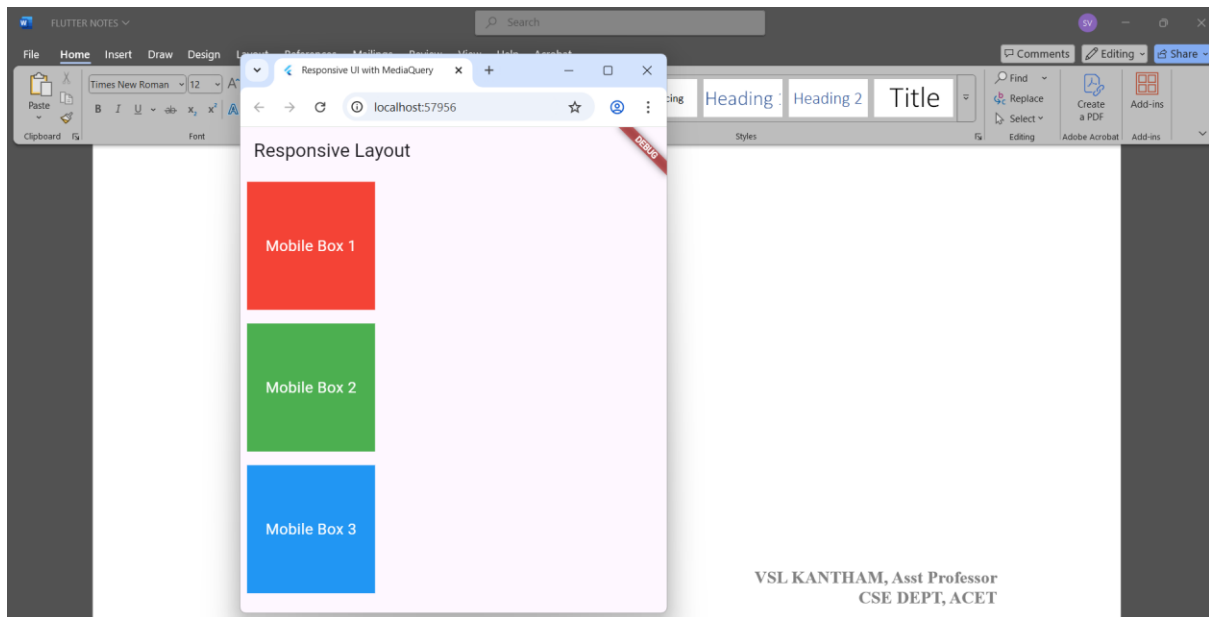
// 🖥 Desktop layout (row)
Widget _buildDesktopLayout() {
    return Row(
        mainAxisAlignment: MainAxisAlignment.spaceEvenly,
        children: [
            _buildBox(Colors.red, 'Desktop Box 1'),
            _buildBox(Colors.green, 'Desktop Box 2'),
            _buildBox(Colors.blue, 'Desktop Box 3'),
        ],
    );
}

// 🔧 Reusable widget
Widget _buildBox(Color color, String label) {
    return Container(
        width: 150,
        height: 150,
        margin: EdgeInsets.all(8),
        color: color,
        child: Center(
            child: Text(
                label,
                style: TextStyle(color: Colors.white, fontSize: 18),
            ),
        ),
    );
}
```

```
),  
,  
);  
}  
}
```

OUTPUT:





VSL KANTHAM, Asst Professor
CSE DEPT, ACET

4 a) Set up navigation between different screens using Navigator.

- Navigator.push moves to a new screen
- Navigator.pop returns to the previous screen

```
import 'package:flutter/material.dart';
```

```
void main() => runApp(MyNavigationApp());
```

```
class MyNavigationApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Navigator Demo',  
      theme: ThemeData(primarySwatch: Colors.pink),  
      home: HomeScreen(),  
    );  
  }  
}
```

```
// 🏠 Home Screen
```

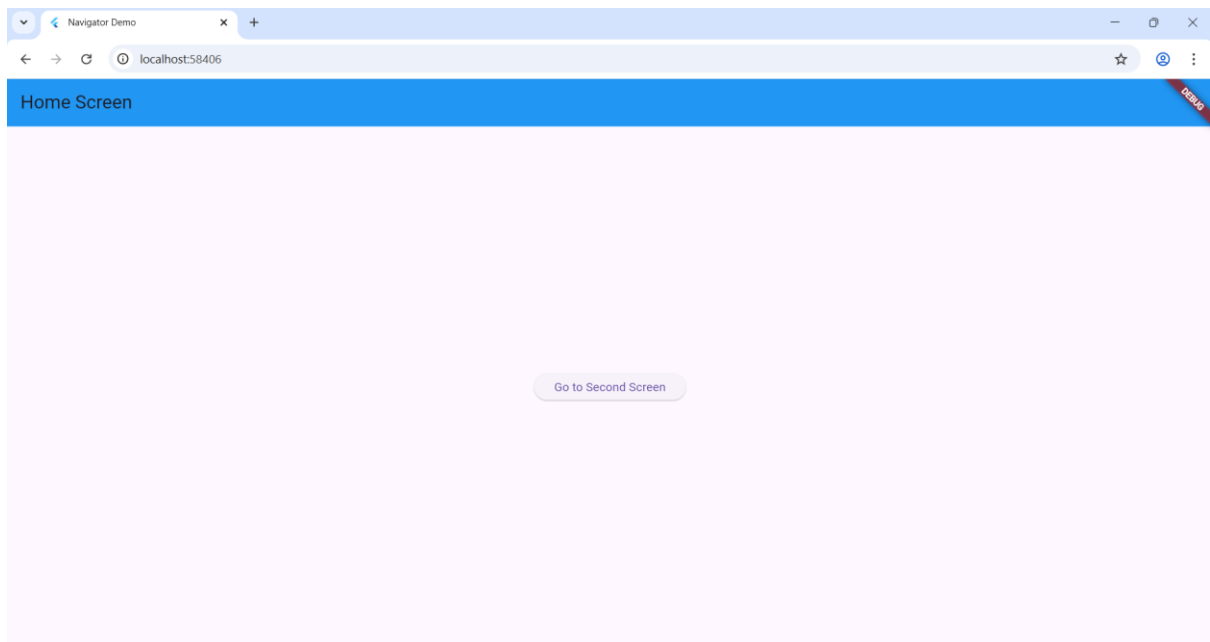
```
class HomeScreen extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: Text('Home Screen'), backgroundColor: Colors.blue),  
      body: Center(  
        child: ElevatedButton(  
          child: Text('Go to Second Screen'),  
          onPressed: () {  
            Navigator.push(  
              context,  
              MaterialPageRoute(builder: (context) => SecondScreen()),  
            );  
          },  
        ),  
      ),  
    );  
  }  
}
```

```
    ),  
    );  
  }  
}
```

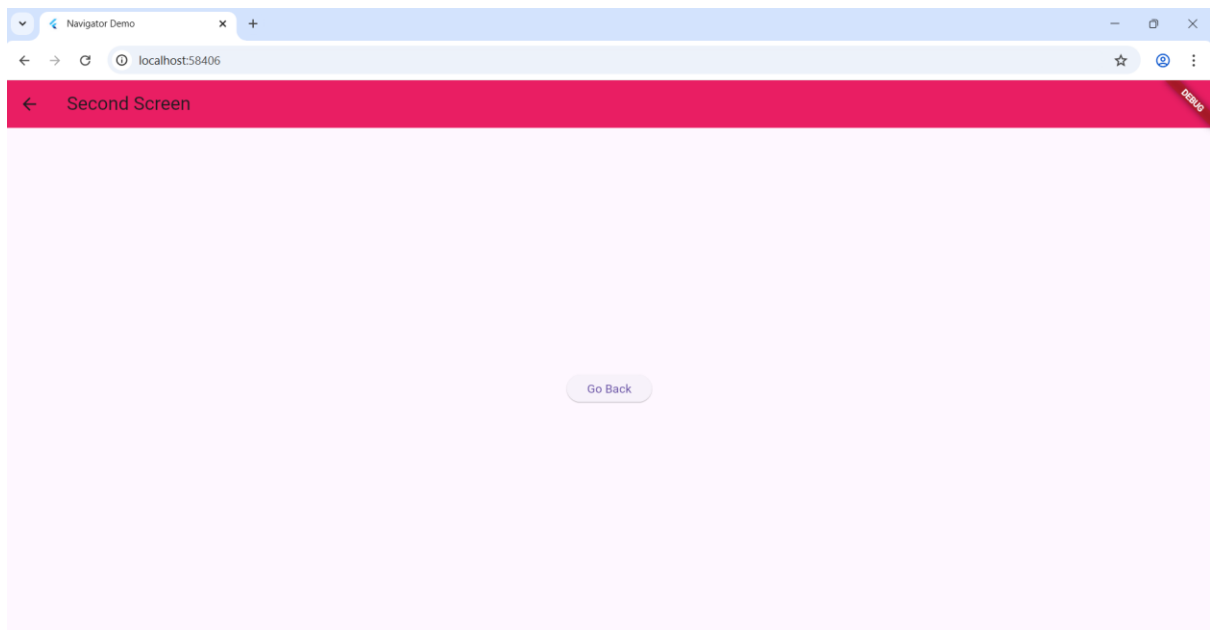
//  Second Screen

```
class SecondScreen extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text('Second Screen'),  
        backgroundColor: Colors.pink,  
      ),  
      body: Center(  
        child: ElevatedButton(  
          child: Text('Go Back'),  
          onPressed: () {  
            Navigator.pop(context);  
          },  
        ),  
      ),  
    );  
  }  
}
```

OUTPUT:



FIRST SCREEN



SECOND SCREEN

b) Implement navigation with named routes.

```
import 'package:flutter/material.dart';
```

```
void main() => runApp(MyNamedRouteApp());
```

```
class MyNamedRouteApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Named Route Demo',  
      // ✅ Define named routes here  
      initialRoute: '/',  
      routes: {  
        '/': (context) => HomeScreen(),  
        '/second': (context) => SecondScreen(),  
      },  
    );  
  }  
}
```

```
// 🏠 Home Screen
```

```
class HomeScreen extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: Text('Home Screen')),  
      body: Center(  
        child: ElevatedButton(  
          child: Text('Go to Second Screen'),  
          onPressed: () {  
            // 🚀 Navigate using named route  
            Navigator.pushNamed(context, '/second');  
          },  
        ),  
      ),  
    );  
  }  
}
```

```

    ),
  ),
);
}
}

```

//  Second Screen

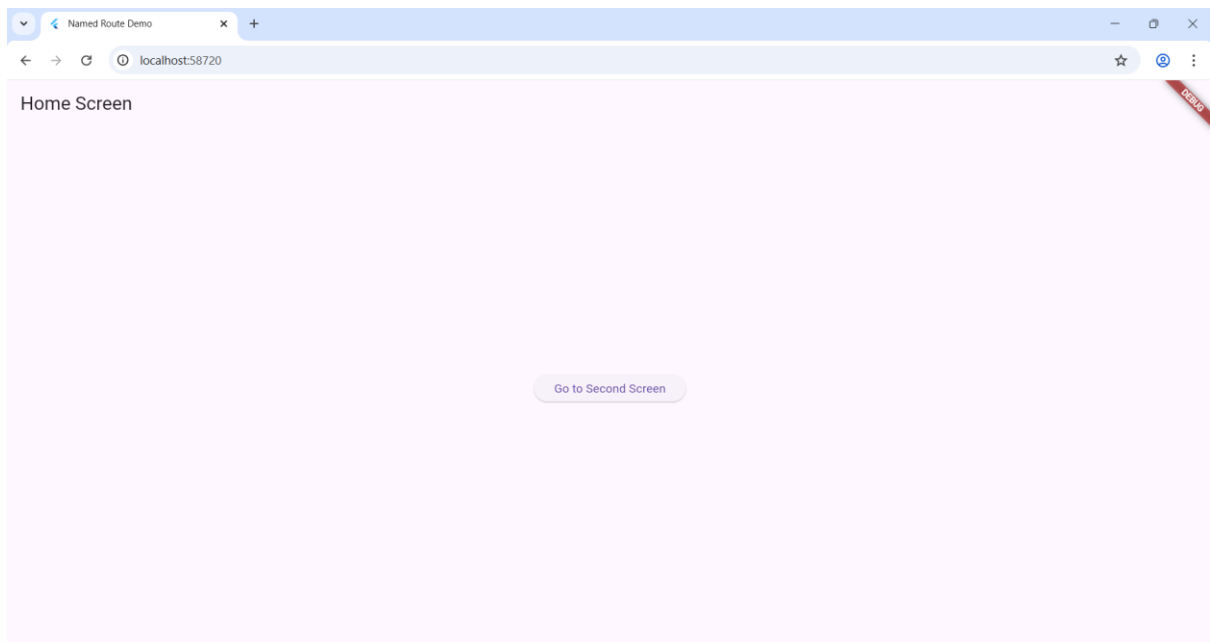
```

class SecondScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Second Screen')),
      body: Center(
        child: ElevatedButton(
          child: Text('Go Back'),
          onPressed: () {
            Navigator.pop(context); //  Go back
          },
        ),
      ),
    );
  }
}

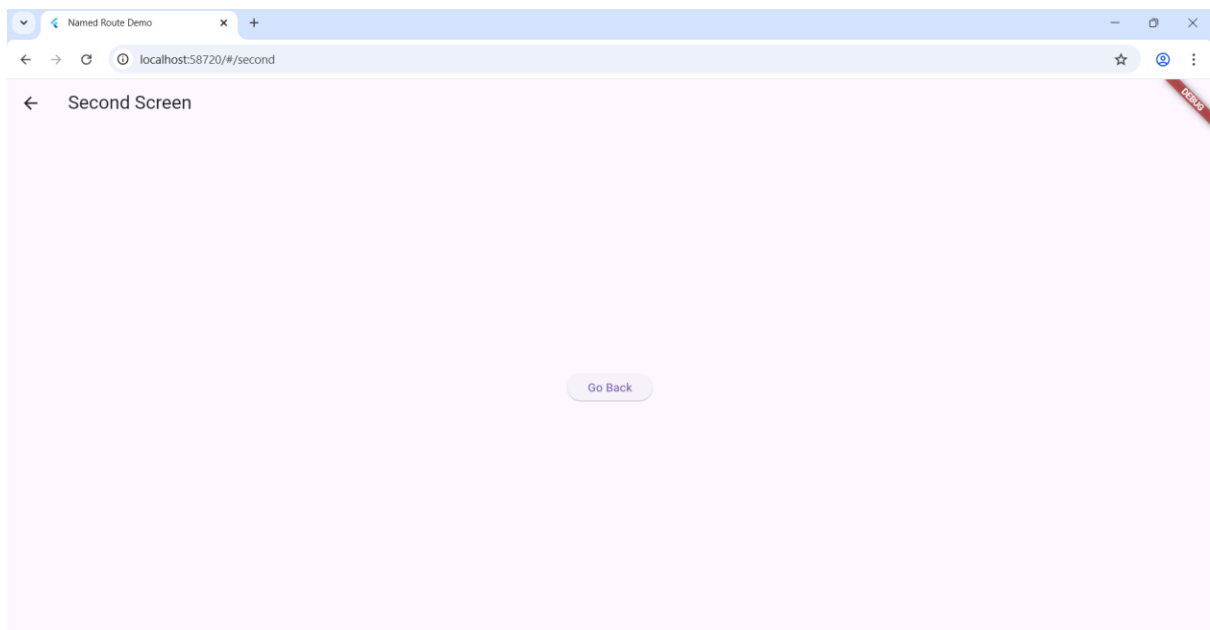
```

Concept	Description
routes	A map of route names to widget builders
initialRoute	The first screen shown when the app starts
Navigator.pushNamed	Navigates using a route name
Navigator.pop	Returns to the previous screen

OUTPUT :



FIRST SCREEN



SECOND SCREEN